

Contract Vault

Front-End Connection & Architecture Technical Specification

Executive Summary: This document provides an exhaustive breakdown of the working, architecture, and integration of the **React Front-End Interface (web2)** for the Anti-Tamper Legal & Business Contract Vault. It details how client-side user interactions trigger SHA-256 cryptographic hashing, communicate with the Express backend REST service, pin files onto IPFS (Pinata), and verify/anchor smart contract state on the Polygon Amoy blockchain network.

1. High-Level Front-End System Architecture

The React client operates as a zero-trust presentation layer that delegates heavy cryptographic signatures and node connectivity to the dedicated backend service while maintaining real-time reactive UI state for the user.

Component Layer	Technology	Primary Responsibility
User Interface	Vite + React 18	Drag-and-drop document upload, live verification feedback UI.
Service Abstraction	Fetch API / Vite Proxy	Wraps HTTP endpoints (/api/*) and manages multipart form data.
State Management	React Hooks (useState/useEffect)	Tracks tab navigation, health polling, loading states & hashes.
Backend REST API	Node.js + Express	Handles SHA-256 hashing, Pinata IPFS pinning & Polygon web3 interaction.
Smart Contract	Solidity (ContractVault.sol)	On-chain record vault stored on Polygon Amoy Testnet.
Storage Gateway	IPFS / Pinata Gateway	Decentralized storage for document verification metadata.

2. Front-End Code Structure & File Connections

The /web2 directory contains the React application structured as follows:

- **src/main.jsx:** Mounts the root App component into the HTML DOM root container.
- **src/App.jsx:** Top-level component orchestrating Header status badges, Hero banner, StatsBar, tab switching (Upload vs Verify), and background ambient animations.
- **src/services/api.js:** Centralized HTTP API client encapsulating all backend REST communication.
- **src/components/UploadPortal.jsx:** Controls file selection, dropzone state, multi-step progress bar (Hash → IPFS → Anchor), and success payload display.
- **src/components/VerificationDashboard.jsx:** Handles document verification uploads, rendering instant **BOLD GREEN** (authentic) or **BOLD RED** (tampered) alert cards.
- **src/index.css:** Complete Design System containing custom tokens, aurora background canvas, shimmer animations, and glassmorphism styling.

3. Service Layer Working (api.js & Vite Proxy)

All outgoing HTTP communication is centralized in `src/services/api.js`. The frontend relies on relative paths (e.g., `/api/anchor`), which Vite proxies to `http://localhost:4000` in development, eliminating CORS complexity.

A. Health Check Connection (`checkHealth`)

The frontend polls `GET /api/health` every 30 seconds to update the header status badge ('Live' vs 'Offline') and every 5 seconds to update the total anchored document counter in the StatsBar.

B. Document Hashing & Anchoring Workflow (`anchorDocument`)

When a user drops a PDF or contract in `UploadPortal.jsx`, the client constructs a `FormData` object appending the raw binary file:

```
// api.js - Anchoring API Call
export async function anchorDocument(file) {
  const form = new FormData();
  form.append('document', file);

  const res = await fetch('/api/anchor', {
    method: 'POST',
    body: form,
  });
  if (!res.ok) throw new Error('Anchoring failed');
  return res.json(); // returns { hash, ipfsCid, txHash, blockNumber, explorerUrl }
}
```

4. Verification & Anti-Tamper Connection Mechanics

Verification is the core security feature of Contract Vault. The connection flow ensures that modified documents trigger immediate tamper detection.

Step-by-Step Verification Flow:

- 1. File Submission:** User drops a document into VerificationDashboard.jsx.
- 2. Binary Stream API Call:** Invokes verifyDocument(file) via POST /api/verify.
- 3. Local Re-Hashing:** Backend server computes SHA-256 fingerprint in-memory without saving to disk.
- 4. Blockchain Query:** Backend calls ContractVault.verifyDocument(_hash) on Polygon testnet.
- 5. Mismatch Evaluation:** Backend returns JSON payload specifying { exists, verified, anchoredDate, anchoredBy, txHash }.
- 6. Reactive UI Render:**
 - **If Verified (exists = true):** Renders VerifiedAlert — Giant green card, green fingerprint box, exact date, wallet address, and PolygonScan link.
 - **If Tampered (exists = false):** Renders TamperedAlert — Giant red alert card, red warning box, computed hash, and tamper warning explanation.

5. Front-End to Back-End API Endpoint Contract Table

Endpoint	HTTP Method	Request Payload	Front-End Component	Response Key Data
/api/health	GET	None	App.jsx (Header & Stats)	status, anchored (count)
/api/hash	POST	FormData (file)	api.js service helper	hash, filename, sizeBytes
/api/anchor	POST	FormData (file)	UploadPortal.jsx	hash, ipfsCid, txHash, blockNumber
/api/verify	POST	FormData (file)	VerificationDashboard.jsx	verified, exists, timestamp, owner

6. Security & Key Isolation Architecture

- **Zero Private Key Exposure:** Web3 private keys (Alchemy/Polygon deployment wallet) and IPFS API JWT credentials (Pinata) remain strictly isolated inside backend/.env. The React frontend makes pure REST calls.
- **In-Memory File Processing:** Uploaded files are processed in-memory via Express Multer memory storage buffers. Files are hashed and discarded without being logged or retained on server disk.
- **Deterministic Fingerprinting:** Any single character change in a contract drastically alters the resulting 256-bit SHA-256 hash (Avalanche effect), rendering tampering mathematically detectable.

7. Verified Live Trial Results

During system validation, 11 end-to-end integration tests were executed against real legal contract files:

Trial #	Test Feature	Test File Input	Front-End Render Outcome
---------	--------------	-----------------	--------------------------

1	Health Monitoring	System Ping	Status badge → GREEN 'Live', Counter = 4
2 & 3	SHA-256 Hashing	NDA & Service Agreement	Hashes computed accurately in < 50ms
4 & 5	Blockchain Anchoring	NDA Agreement PDF	TX 0xa0fbe9... Block #50,311,741 on Polygon Amoy
6	Original Verification	Authentic NDA PDF	■ BOLD GREEN ALERT — Verified & Authentic
7	Tamper Detection	Modified NDA (clause change)	■ BOLD RED ALERT — Cryptographic Mismatch
8	Duplicate Guard	Re-anchoring NDA PDF	HTTP 409 Conflict — Guard Triggered

Document Author: Anti-Tamper Legal Vault Engineering Team | Date: September 2026